M&S Final Assignment: 3DoF Robot Arm

Group 12: J. Buursink (s2790882)
R. Caballero Cano (s3739333)
V. Malaxa (s2726254)
S. Srinivasan (s3603288)

June 21, 2026

1 Introduction

This report presents the modeling, simulation, and control of a 3 Degrees of Freedom (DoF) robotic arm with rotational joints in a 3D space. For this project, inspiration is taken from the PUMA 260 and PUMA 560 robots to make the proportions of the robot realistic. Although the actual PUMA 260 and 560 are 6 DoF robots, we have focused on the first three joints (the waist, shoulder and elbow) to meet the requirements of the final assignment.

The main goal of this project is to combine everything that has been learned throughout the course about physics, modeling and control systems. For example, instead of using a simplified stiff shaft motor model, the system includes full actuator dynamics and Series Elastic Actuators (SEAs). And to drive the motors, a discrete time digital control architecture has been designed that uses cascaded loops for seperate current and torque control.

For the simulation, the robot arm must perform a specific motion task in 3D. To manage this movement and handle physical interactions safely we implemented a Cartesian impedance control. Additionaly, we included gravity compensation to ensure the arm can support its own weight and such that the arm can behave accurately like a spring and damper irrespective of its configuration.

A major focus of this report is proving that our simulation is realistic and well verified. In the following sections, we explain our control formulas (current, torque, impedance, and gravity compensation) and show how we selected realistic parameters.

2 Model

2.1 Theoretical background

2.1.1 Series Elastic Actuator

The core behaviour of a Series Elastic Actuator (SEA) relies on the mechanical compliance placed between the gearbox output and the rigid robot link. Instead

of a rigid connection, a spring is used to transmit torque and absorb sudden external impacts. This spring can also be used to measure the torques.

The torque transmitted to the robot link depends on the deflection of this elastic element. The equation of the spring is defined as:

$$\tau_{\text{spring}}(t) = K_{\text{sea}} \left(\theta_m(t) - \theta_{\text{link}}(t)\right) \tag{1}$$

where K_{sea} is the spring stiffness, The term $\theta_m(t)$ represents the output position of the gearbox and thus the full motor structure, and $\theta_{\text{link}}(t)$ is the absolute position of the rigid robot link.

This mechanical configuration ensures that the force applied to the robot is a direct function of the spring deformation. Therefore, by knowing the spring stiffness and measuring the deflection, the torques can be accurately measured. As well as this, for lower spring stiffnesses the SEA serves to decouple the motor shaft from the actual motion of the joint link.

2.1.2 Impedance Controller

An impedance controller is a controller that generates a force based on a desired physical behaviour. In this system for this a spring-damper impedance controller will be considered. The constitutive relation of a spring was given previously. There we see that a stiffness determines the force caused by a certain deflection. In the impedance controller this is done by placing a virtual spring between the end effector and the desired position. Where the virtual deflection is the difference in distance between those points. For the virtual damping this works in a similar way. Where the damping force is proportional to the end effector velocity with the virtual damping. So we get the following law for impedance control:

$$F_{\text{des}}(t) = K_{\text{stiff, virt}} \left(x_{des} - x_{\text{ee}}(t)\right) + K_{damp,virt}\dot{x}_{ee} \tag{2}$$

where $K_{stiff,virt}$ is the virtual stiffness and K_{damp} is the virtual damping. F_{des} is the resulting desired end effector force. x_{des} is the desired end effector position, and x_{ee} is the actual end effector position.

2.1.3 Cascaded Control Loop Theory

To regulate the electrical and mechanical dynamics of the motor a cascaded control architecture was used. This structure separates the control task into two parts: an outer loop for torque and an inner loop for current.

The outer loop minimizes the error between the desired joint torque $\tau_{\text{des}}(t)$ and the measured spring torque $\tau_{\text{spring}}(t)$. A Proportional Integral (PI) control law calculates the required reference current $I_{\text{des}}(t)$ the system should have to achieve the desired torque:

$$I_{\text{des}}(t) = K_{p,\tau} \cdot e_\tau(t) + K_{i,\tau} \int_0^t e_\tau(\tau)\, d\tau \tag{3}$$

where $e_\tau(t) = \tau_{\text{des}}(t) - \tau_{\text{spring}}(t)$, and $K_{p,\tau}, K_{i,\tau}$ are the continuous-time torque controller gains.

The inner control loop tracks this reference current by checking the current error $e_i(t) = I_{\text{des}}(t) - i(t)$. A fast PI control law deternmines the required motor voltage $V(t)$:

$$V(t) = K_{p,i} \cdot e_i(t) + K_{i,i} \int_0^t e_i(\tau)\, d\tau \tag{4}$$

where $K_{p,i}$ and $K_{i,i}$ are the current controller gains.

In control theory, the stability of a cascaded system relies on the frequency separation principle. The inner current loop must be tuned to have a much higher bandwidth than the outer torque loop. Because the electric dynamics are faster than the mechanical springs dynamics, the current reaches its reference value almmost instantly. Therefore, the current can be assumed to match its reference reference ($i(t) \approx I_{\text{des}}(t)$) when analyzing the slower torque tracking loop

2.1.4 Jacobian Matrix Derivation

The Jacobian matrix $\mathbf{J}(\boldsymbol{\theta})$ is a fundamental linear transformation that relates the joint velocities to the Cartesian velocities of the end-effector by:

$$\dot{x} = J\dot{q} \tag{5}$$

And the end effector wrench to the joint torque by:

$$\tau = J^T F \tag{6}$$

The jacobian was derived analytically by calculating the partial derivatives of the forward kinematics equations with respect to each joint angle ($J_{ij} = \frac{\partial P_i}{\partial \theta_j}$):

$$\mathbf{J}(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} & \frac{\partial x}{\partial \theta_3} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} & \frac{\partial y}{\partial \theta_3} \\ \frac{\partial z}{\partial \theta_1} & \frac{\partial z}{\partial \theta_2} & \frac{\partial z}{\partial \theta_3} \end{bmatrix} \tag{7}$$

To make it easer to write on 20-sim and avoid, three auxiliary variables ($\sigma_1, \sigma_2, \sigma_3$) were introduced in the code:

$$\sigma_3 = L_3 \sin(\theta_2 + \theta_3) \tag{8}$$
$$\sigma_2 = \sigma_3 + L_2 \sin(\theta_2) \tag{9}$$
$$\sigma_1 = L_3 \cos(\theta_2 + \theta_3) + L_2 \cos(\theta_2) \tag{10}$$

Substituting these variables into the partial derivatives, the resulting 3×3 Jacobian matrix executed within the 20-sim model is defined as:

$$\mathbf{J}(\boldsymbol{\theta}) = \begin{bmatrix} -\sin(\theta_1)\sigma_2 & \cos(\theta_1)\sigma_1 & L_3 \cos(\theta_2 + \theta_3)\cos(\theta_1) \\ \cos(\theta_1)\sigma_2 & \sin(\theta_1)\sigma_1 & L_3 \cos(\theta_2 + \theta_3)\sin(\theta_1) \\ 0.0 & -\sigma_3 - L_2 \sin(\theta_2) & -\sigma_3 \end{bmatrix} \tag{11}$$

2.1.5 Gravity Compensation

Because the manipulator operates in a 3D spatial environment, the links experience significant configuration-dependent gravitational torques. To ensure that the robot can maintain its posture without errors and to allow the Cartesian impedance controller to handle purely external interactions, an analytical gravity compensation vector $\mathbf{G}(\mathbf{q}) = [g_1, g_2, g_3]^T$ was derived and implemented.

Based on the kinematic configuration and mass distribution of the developed 3 DoF robot, the gravitational torques for each joint are :

$$g_1 = 0.0 \tag{12}$$

$$g_2 = g\left(\frac{m_2 l_2}{2} + m_{c1} l_2 + m_3 l_2 + m_{c2} l_2\right)\cos(\theta_2) + g\left(\frac{m_3 l_3}{2} + m_{c2} l_2\right)\cos(\theta_2 + \theta_3) \tag{13}$$

$$g_3 = g\left(\frac{m_3 l_3}{2} + m_{c2} l_3\right)\cos(\theta_2 + \theta_3) \tag{14}$$

Where:

- g represents the standard acceleration due to gravity (`grav`).
- θ_2 and θ_3 denote the angular positions of the shoulder and elbow joints, respectively (`theta2`, `theta3`).
- m_2 and m_3 are the nominal masses of the second and third links, with their centers of mass assumed to be at the geometric half length ($l_2/2$ and $l_3/2$).
- m_{c1} and m_{c2} represent the additional concentrated lumped masses (`mc1`, `mc2`) positioned at the joint interfaces, which directly account for the physical weight of the AK10-9 actuators.

The physical meaning behind these equations reflects the system kinematics. The first joint (g_1) requires zero gravitational torque because its axis of rotation is perfectly aligned with the vertical gravity vector, so changes in the waist angle do not alter the gravitational potential energy of the arm.

Conversely, the shoulder (g_2) and elbow (g_3) joints rotate about horizontal axes, making them highly sensitive to the arm posture. The expression for g_2 compensates for the total moment generated by both links, as well as the actuator masses acting at their respective lever arms (l_2 and l_3). The expression for g_3 isolates the torque required to support the third link and its corresponding joint actuator.

2.2 Model Overview and 20-sim Submodel Implementation

This section explains how the theoretical equations are implemented in 20-sim. The global layout of the system is presented, and the function of each important block is explained.

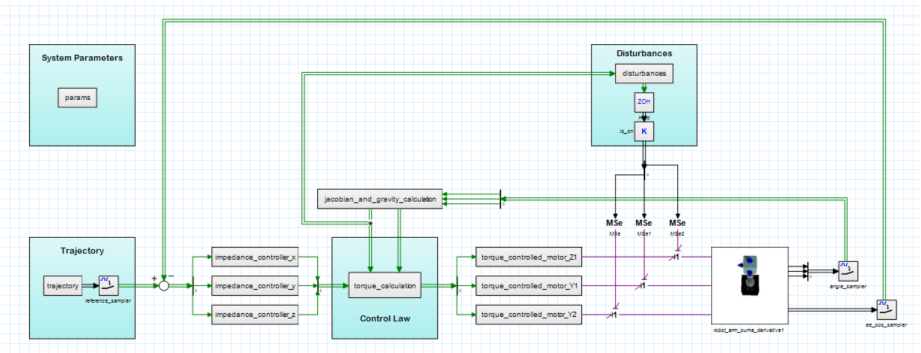


Figure 1: Complete 20-sim block diagram implementation of the 3 DoF robot. Model can be found in G12_FULL_IMPLEMENTATION_PUMA_derivative.emx.

The complete layout of the system is shown in Figure 1. The simulation connects signal lines for data and power bonds for physical interactions between the components. The top-level diagram consists of the following key parts:

- **Trajectory Generation:** This block calculates and provides the reference coordinates for the path.
- **Impedance Controllers:** These blocks evaluate the position tracking errors to determine the virtual forces.
- **Torque Calculation, Jacobian, and Gravity Compensation Blocks:** These blocks are responsible for implementing the torque control law.
- **Torque Controlled Motors:** These blocks manage the discrete cascaded loops for the joint motors and models the motors themselves.
- **Parameters block:** This block centralizes all the physical constants and control parameters of the robot. This keeps the model organized and makes it easy to change settings without opening every single submodel.
- **Disturbances:** This block is used to inject external forces into the physical joints. It allows the robustness of the controller to be tested when the robot encounters unexpected interference.
- **Robot Arm:** This block contains the physical robot arm where the motion and link dynamics are simulated.

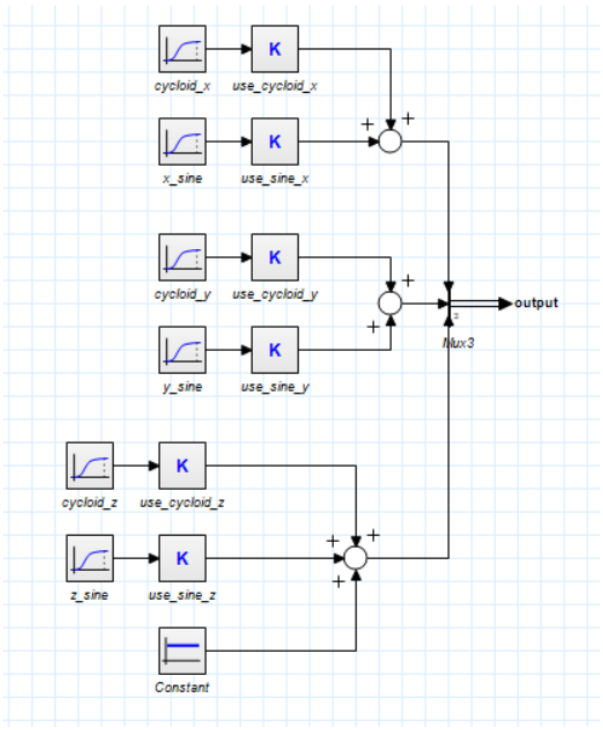


Figure 2: Block diagram implementation of the trajectory generation subsystem.

The path generator block is shown in Figure 2. This block calculates the 3-D target trajectory that the robot needs to follow. Smooth positions $\mathbf{X}_{\text{des}}(t)$ are output over time to avoid sudden changes, as the controllers would need to output high torques and currents to track what are references with an effectively infinitely high slope due to the discontinuities.

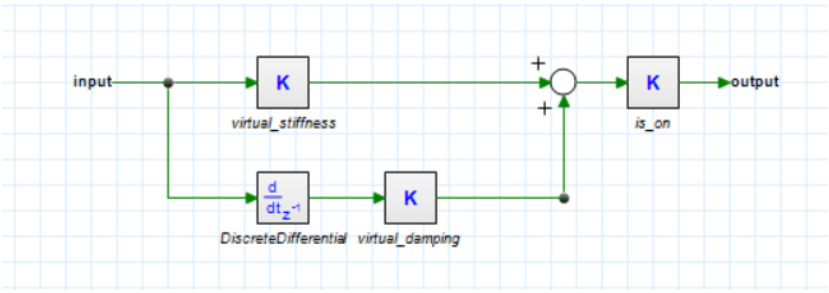


Figure 3: Block diagram implementation of the impedance controller subsystem

The impedance controller block is detailed in Figure 3. The target trajectory is compared with the actual position of the end-effector. Then, virtual stiffness and damping gains ($\mathbf{K}_p$ and $\mathbf{K}_d$) for the x, y, and z axes are used to calculate the required virtual force vector $\mathbf{F}_{\text{in}}$.

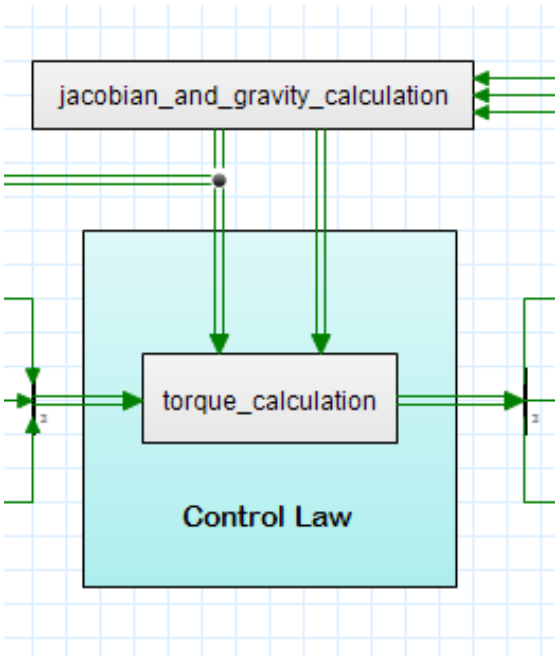


Figure 4: Block diagram of the control law, jacobian, and gravity compensation calculations.

In jacobian_and_gravity_calculation in Figure 4. The angles of the joints are used to calculate the jacobian and experienced gravity at each joint. With these calculated the torque_calculation block then uses the following control law which is then sent to the controlled motors:

$$\tau_{des} = J^T F + \mathbf{G}(q) \tag{15}$$

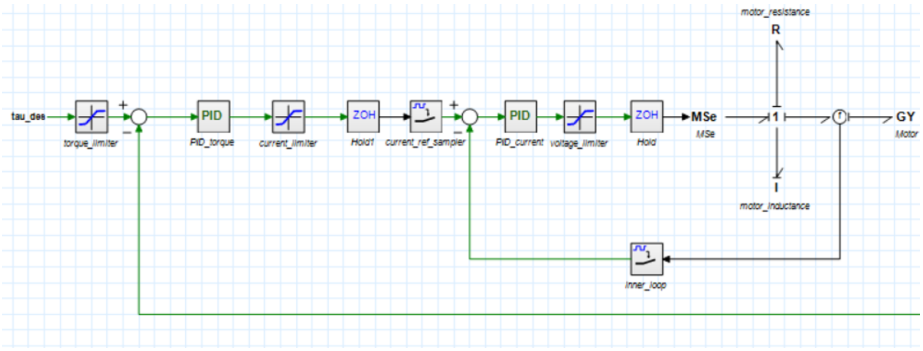


Figure 5: Block diagram implementation of the torque controller subsystem Part 1.

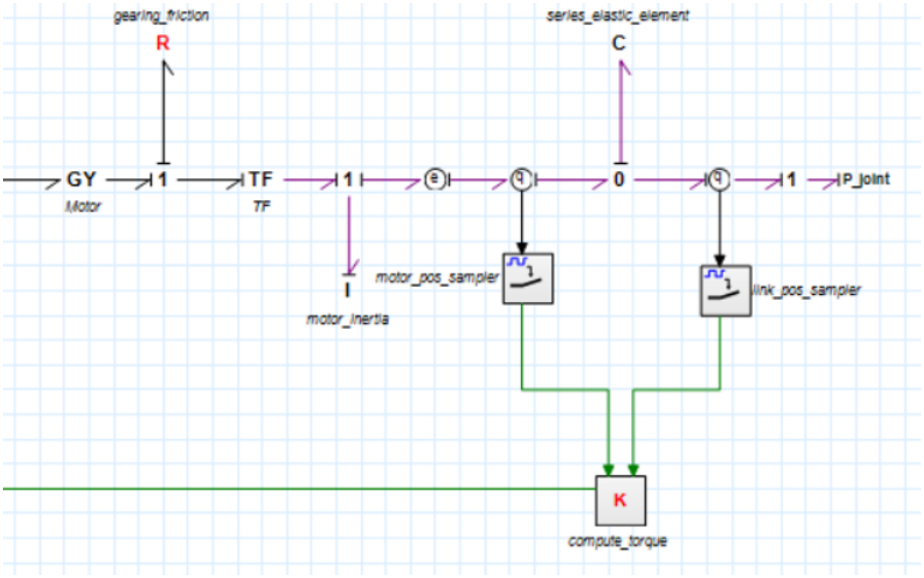
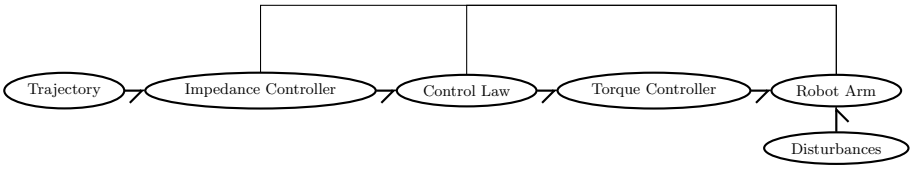


Figure 6: Block diagram implementation of the torque controller subsystem Part 2.

Finally, Figure 5 and Figure 6 shows the internal layout of the torque controlled motors. This block works digitally at a sampling frequency of 10000 Hz for the electrical parts and 1000 Hz for the mechanical parts. The desired joint torques from the gravity and Jacobian mapping are received, and a cascaded loop is executed: the outer PI loop measures the torque using the spring deflection and then uses a PI controller to find the target current I_{des}. The inner PI loop controls the current that the outer PI controller requests by controlling the voltage V the motor receives.

A broad overview of the system configuration and the interconnection between the submodels is presented in the word-based diagram below:



The interaction between the components follows a sequential flow:

- **Trajectory:** The desired position coordinates are calculated and sent as inputs to the controller.
- **Impedance Controller:** The reference path is compared with the actual coordinates to generate the virtual forces.
- **Torque Calculation, Jacobian, and Gravity Compensation:** The virtual forces are together with the jacobian and gravity terms are turned into desired torques for the motors.

- **Torque Controller:** The joint actuator loops are managed here.
- **Robot Arm:** The physical dynamics of the robot arm are simulated in this block. External inputs from the **Disturbances** block are applied here to evaluate the robustness of the system against unexpected forces.

2.3 Modelling choices

To achieve accurate trajectory tracking, the operational space position of the end-effector is continuously compared against the desired reference profile. Joint motion is driven by a computed torque control strategy comprising two primary components: a feedforward gravity compensation vector derived analytically (as detailed in Eqs. 12–14), and a feedback joint torque vector mapped from the operational space using the transpose of the manipulator Jacobian matrix ($\mathbf{J}^T$). This feedback torque is generated by processing the Cartesian tracking errors through independent operational-space impedance controllers across each coordinate axis. The specific modeling assumptions, architectural simplifications, and subsystem implementations selected for the 20-sim environment are detailed in the following subsections.

2.3.1 Trajectory

Trajectory generation defines the reference motion inputs for the 3 DoF manipulator. This section outlines the specific assumptions and modeling choices selected for path planning within the simulation.

- The general trajectory that is used for testing starts with a cycloid input which reaches a desired constant position where all x, y, and z are no longer at their start positions and then after some time the trajectory switches to a circular motion in the XZ plane.
- **Smooth Trajectory Profiles:** References are generated using smooth mathematical curves instead of step steps. This ensures continuity in all reference curves velocity and acceleration, which avoids the controllers from obtaining large spikes due to infinitely high slopes which are inputs to the derivative terms such as the damping term in the impedance controllers.
- **Rest-to-Rest Boundary Conditions:** Initial and final velocities and accelerations are set strictly to zero. This ensures that all movements start and end from a complete standstill, eliminating sudden inertial shocks to the robot structure.

2.3.2 Parameter Block

A dedicated parameter block is implemented to centralize all physical, geometric, and control variables of the system. This choice allows for the rapid modification of parameters from a single location, eliminating the need to alter individual submodels during simulation trials. And it also serves to keep a simple centralized overview of all parameters.

2.3.3 Impedance controller

To regulate the manipulator compliance and interaction behavior, an impedance control strategy is implemented based on the following modeling choices:

- **Discrete PD Architecture:** The impedance controller is modeled as a discrete parallel Proportional Derivative (PD) structure, representing a virtual spring-damper system. As illustrated in Figure 3, a discrete differential block is chosen to derive the velocity term from the input signal, avoiding the need for continuous time derivatives as in practice there is no option for continuous computation and especially not at these speeds.
- **Activation Management:** A digital gain switch is incorporated directly into the controller output path. This choice allows the impedance control actions to be enabled or disabled during specific simulation trials without modifying the underlying structure, or stiffness and damping parameters.
- **Virtual Stiffness Parameter Choice** The impedance controllers have all been chosen to have a stiffness of 200 [N/m] this is calculated by considering the testcase of a person's hand getting stuck between the robot arm and the wall. In such a case the robot should be moved enough distance such that the hand could be removed which we consider to be about 5 cm. Then we need to consider what kind of force should be able to move the robot this distance. We consider that a person should be able to move the robot this distance with a low amount of force. We consider low to be around 1 kg of force or about 9.81 N so the stiffness required for this case is the following:

$$K_{stiff,virt} = \frac{F}{d} = \frac{9.81}{0.05} = 196Nm \approx 200Nm \tag{16}$$

  We round up to 200 to make testing easier in the model since we can then use nice factors of 10 for the forces and desired deflections. While possible to make the virtual stiffness can be different in each cartesian direction right now there is no reason to do so as the task does not ask for that. However, it might be preferred to have a higher z stiffness if the task is for example to pick up items of different weights such that the robot can have a similar z position for each different load weight, while the x and y stiffness could stay the same such that the robot can be moved out of the way of harm. But for now the virtual stifnesses are the same in each cartesian direction as this is not an explicit task of the robot.
- **Virtual Damping Parameter Choice** The damping parameters are chosen through educated trial and error. There is a relation between stiffness and damping that causes a system to be either undamped (with low damping), overdamped with (high damping) or critically damped (with the right amount of damping). These situations are signified by oscillations, not reaching the target outcome, and a best possible outcome respectively. The damping has been tuned by trail and error to see what gives the best outcome that does not oscillate, but also reaches a steady state in a reasonable timeframe. This way the damping in each cartesian direction is chosen to be 50 Ns/m.

2.3.4 Torque controller

To achieve precise joint torque regulation, an identical actuation model is replicated across all three rotational joints of the manipulator. The physical behavior and control architecture are defined inside a unified submodel composed of two cascaded loops, as illustrated in Figure 5 and Figure 6. The modeling choices, parameters, and calibration methods are detailed below:

Cascaded Control Architecture and Loop Design

- **Inner Electrical Loop:** The core of the actuator represents the physical electric motor that generates electromagnetic torque. Within this block, the motor inductance is implemented as an inertia (I) element of which the value is taken from the datasheet of the CubeMars AK10-9 V2.0 KV60 robotic actuator at 0.000181 H [1], and the internal electrical losses are modeled as a resistive (R) element. Voltage regulation within this inner loop is executed at a high sampling rate of 10 kHz this high frequency is necessary for the controller to be significantly faster than the torque controller such that the currents required for the torque generation can be achieved in time. The voltage that sets this current is governed by a strict hardware saturation limiter of 48 V of which the value is also given in the datasheet of the motor.
- **Outer Torque Loop:** Current control is achieved using a discrete Proportional-Integral (PI) tracking architecture, which is restricted by a maximum current limiter of 29.8 A. The outer loop incorporates the structural dynamics of the integrated gearbox—accounting for its intrinsic inertia and a combined friction profile consisting of both viscous and Coulomb friction components. Torque estimation is continuously computed through a Series Elastic Actuator (SEA) block. The final output of the torque loop is bounded by a maximum protective constraint of 48 $\mathrm{N} \cdot$ m.
- **Controller Parameters** For both control loops the choice is made to use a PID controller over a PI controller. This is because there was a lot of high frequency oscillation in the controller outputs despite setting the D element low. So the choice was made to exclude it in its entirity, making a PI controller. The values of the PID controller have been chosen by educated trial and error. Having a high I term which in the model is a low integral time constant. Can cause the model to overcompensate and thus explode. The same goes for the proportional controller, but that serves a more direct change in output whereas integral controller output needs to build up over time. Doing this educated trial and error tuning gave the following PI tuning parameters for the torque controller:
  - $K_p = 1$
  - $\tau_I = 0.02$

  And for the current controller these are:
  - $K_p = 1$
  - $\tau_I = 0.0005$

Empirical Parameter Verification and Resistance Calibration

- All baseline physical parameters are sourced from the official documentation of the CubeMars AK10-9 V2.0 KV60 robotic actuator and verified using the manufacturer's performance analysis charts [1].
- While the majority of nominal values were implemented as provided, the winding resistance required an empirical adjustment. The manufacturer specifies a nominal phase-to-phase resistance of 0.195 Ω; however, this baseline value proved insufficient to capture real-world operating losses within the simulation environment.
- To align the model with experimental data, the effective operating resistance was recalculated at a specific torque benchmark of 50 N · m. At this operating point, the angular speed after the gearbox was extracted from the manufacturer's performance chart. By applying the gear reduction ratio and multiplying by the torque constant (K_t), the back-electromotive force (back-EMF) of the motor was determined to be approximately 24 V.
- Given a constant nominal voltage supply of 48 V, the remaining potential drop of 24 V acts directly across the internal resistance of the motor wiring. Since the chart indicates a current draw of 38 A at a torque of 50 N · m, the true winding resistance (R_{motor}) is estimated via Ohm's law:

$$R_{\text{motor}} = \frac{V_{\text{supply}} - V_{\text{back-EMF}}}{I_{\text{chart}}} = \frac{48\ \text{V} - 24\ \text{V}}{38\ \text{A}} \approx 0.63\ \Omega \tag{17}$$

- The motor friction values were obtained from the analysis chart and backdrive parameters. The friction of the motor shaft was modelled before the transformer block and hence reduced by a factor of the reduction ratio.
- The Coulomb friction (d_c) parameter was filled in by place by the backdrive torque of the motor ($d_c = \frac{0.8N.m}{9} = 0.088N.m$). The viscous friction (d_v) was calculated from the analysis chart by calculating the total torque provided by the motor at no load speed and by subtracting the Couloumb friction from it.

$$\begin{aligned} \tau &= \frac{I}{K_t} && = 0.2385N.m \\ \tau_{viscous} &= \tau - d_c && = 0.1505N.m \\ d_v &= \frac{\tau_v}{\omega} && = 0.00045N.m.s/rad \end{aligned}$$

- This did not give the exact expected response. However, by tuning the viscous friction parameter to $0.0001N.m.s/rad$ we were able to get an accurate representation of the motor as the efficiency, rpm, and current vs torque plots, which can be seen Figure 8, matched up relatively accurately with the one in the datasheet which can be seen in Figure 7.

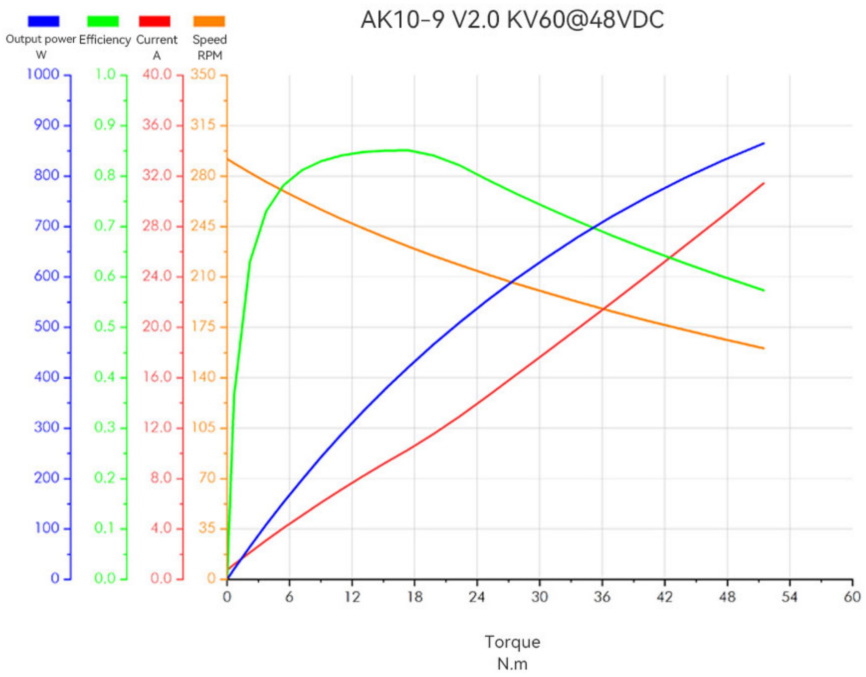


Figure 7: Analysis chart as provided by CubeMars[1]

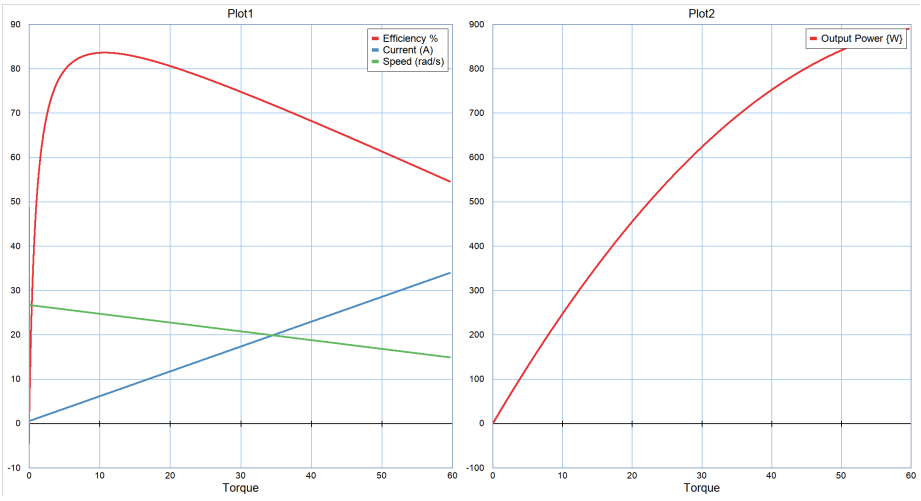


Figure 8: Analysis chart as per our model 'G12_motor_drive_efficiency.emx'

The implementation of these calibrated values (R_{motor}, d_c) ensures that the simulated efficiency, current, power consumption, and RPM curves accurately match the experimental performance metrics provided by the manufacturer.

2.3.5 Series Elastic Actuator (SEA) Stiffness

Within the torque controller submodel, the stiffness of the SEA is set to an arbitrarily high constant value of 10000 $\mathrm{N} \cdot \mathrm{m/rad}$ this is done because the SEA element in the real robot is just the shaft of the motor. The SEA is only used as a torque sensor. In the bond graph, this elastic behaviour is modelled as a C element located between the motor transmission (at the end of the gearbox) and the joint link. By selecting this value, high tracking speed is maintained in the outer torque loop while sufficient deflection is allowed to estimate the joint torque. This estimation is done by comparing the measurements from the motor and link position samplers and then multiplying by the spring stiffness.

2.3.6 Robot Parameters and Actuators Selection

The physical and geometric structure of the 3 DoF manipulator is based on a downscaled PUMA 560 robot configuration. We downscaled the PUMA560 robot as it is quite an old robot and therefore quite heavy with a weight of 83kg [2]. Newer robots such as the UR7e from Universal Robots is only about 20kg and has about 2-3 times the payload capacity at the same reach [3]. So it seems logical that a modern version of the PUMA 560 would be a lot lighter. The PUMA 560 was chosen as a base only because it's weight distribution is publically available. Whereas it is not publically available for most robots. So the choice is made to scale down the PUMA560 to the PUMA260 only in weight while assuming the max payload and spatial dimensions of the PUMA560. The PUMA260 is a version of the PUMA robot which has the same structure as the PUMA560, but was a lighter model with smaller payload and smaller reach at a weight of 13.3 kg [2]. A weight reduction ratio of $\gamma = \frac{13.3}{83}$ is applied to the original PUMA 560 link masses (59.56 kg, 18.4 kg, and 4.8 kg, respectively).

Joint actuation is performed using CubeMars AK10-9 V2.0 KV60 motors. The geometric, mass, and dynamic parameters introduced directly into the 20-sim simulation blocks are summarized in Table 1 and Table 2.

Table 1: Geometric and Scaled Mass Parameters for the PUMA 260 Configuration.

Parameter Description	**Symbol**	**Value**
Geometric Parameters		
Link 1 Length (Base to Shoulder)	L_1	1.0000 m
Link 2 Length (Shoulder to Elbow)	L_2	0.4318 m
Link 3 Length (Elbow to End-Effector)	L_3	0.4331 m
Scaled Link Masses ($\gamma = 13.3/83$)		
Link 1 Mass ($\gamma \times 59.56$ kg)	m_{l1}	9.544 kg
Link 2 Mass ($\gamma \times 18.4$ kg)	m_{l2}	2.948 kg
Link 3 Mass ($\gamma \times 4.8$ kg)	m_{l3}	0.769 kg

Table 2: Inertia Properties Implemented in the 20-sim Model.

Link	**Mass (kg)**	I_{xx} **(kg·m^2)**	I_{yy} **(kg·m^2)**	I_{zz} **(kg·m^2)**
Link 1	9.544	1.00965	1.00965	0.4293
Link 2	2.948	0.088	0.088	0.08496
Link 3	0.769	0.0132	0.0132	0.002464

2.3.7 Disturbance

To evaluate controller robustness and compliance, external disturbances are included in the simulation using the following modelling choices:

- **Gravity:** Gravity is modelled by the 3D-mechanics model of the robot arm.
- **External Force Application:** Environmental interactions and payload changes are modelled as a virtual cartesian force vector acting directly on the end-effector these can be added in the disturbance block of the model.
- **Jacobian Transpose Mapping:** The virtual cartesian force vector is then are transformed into equivalent joint disturbance torques through the transpose of the manipulator Jacobian matrix ($\mathbf{J}^T$).
- **Joint-Level Disturbance Injection:** These disturbance torques are injected directly into the joint axes to simulate non ideal operating conditions in parallel with the main actuator inputs. This is done because we have easy acces to the power bonds representing the joint torques and by the transpose of the jacobian this is the same as applying a force directly to the end effector.

2.3.8 3D Robot Arm

The physical structure of the 3 DoF robot is implemented in 20-sim using the 3D Mechanics Toolbox. This section outlines the specific design choices, coordinate frame assignments, and physical parameters selected to represent the robot arm.

Coordinate Frames and Environment Setup

- **World Reference Frame:** The global origin $(0, 0, 0)$ is positioned at the center of the black box. The coordinate axes are oriented such that the Z-axis points vertically upward and the X-axis points to the left. The first joint, which rotates about the Z-axis, is positioned at $(0, 0, 0.5)$ m.
- **Gravity Configuration:** The gravity vector is defined along the negative vertical axis as $\mathbf{g} = [0, 0, -9.81]^T$ m/s^2 to ensure realistic weight effects on all links.
- **Body-Fixed Frames:** Local coordinate systems are attached to the center of mass of each link, following a consistent orientation to simplify kinematic transformations between the waist, shoulder, and elbow.

Rigid Bodies and Mass Distribution

The robot is modeled as a kinematic chain composed of three independent rigid links. To approximate the physical properties of the system, the mass, inertia, and geometric parameters are based on the PUMA 560 robot configuration which was downscaled to have the smae total mass as the PUMA260. These specific numerical values are detailed in the parameters section (see Section 2.3.6). Based on this configuration, the links are defined as follows:

- **Link 1 (Waist/Base):** Modeled with a physical length of $L_1 = 1.00$ m and a scaled baseline mass of 9.544 kg. The mass and primary dimensions are implemented to represent the structural vertical base pillar of the manipulator, as detailed in Table 1.
- **Link 2 (Shoulder/Upper Arm):** Modeled with a physical length of $L_2 = 0.4318$ m and a scaled baseline mass of 2.948 kg. This link represents the main elevation component, utilizing the corresponding diagonal inertia tensor terms (I_{xx}, I_{yy}, I_{zz}) specified in Table 2.
- **Link 3 (Elbow/Forearm):** Modeled with a physical length of $L_3 = 0.4331$ m and a scaled baseline mass of 0.769 kg. The dynamic distribution captures the forearm characteristics, extending directly to the end-effector position without any added sensor mass penalties.

Joint Definitions and Constraints

- **Joint 1 (Waist):** Modeled as a hinge joint allowing rotation strictly around the local Z-axis. The initial simulation position is set to 0.0 rad.
- **Joint 2 (Shoulder):** Modeled as a hinge joint rotating around the horizontal Y-axis. The initial simulation position is set to 0.0 rad.
- **Joint 3 (Elbow):** Modeled as a hinge joint rotating around the horizontal Y-axis, moving parallel to the shoulder joint. The initial simulation position is set to 0.0 rad.
- **Motion Limits:** Mechanical constraints are implemented for each joint within the simulation model to reflect physical boundaries created in robots such as wire winding. The waist joint is restricted to a motion range of $\pm 180°$. The shoulder and elbow joints are bounded within a range of $\pm 140°$.

Modeling Simplifications and Exclusions:

To make it easier to model, the following simplifications were made:

- **Structural Rigidity:** All mechanical elements are considered perfectly rigid bodies.
- **End-Effector Sensor Mass:** Although a physical sensor and body are attached to the end-effector, its mass is extremely small relative to the rest of the manipulator structure about 1 gram. Consequently, this mass is considered negligible and is omitted from the dynamic equations to simplify the model and the bright red ball mostly serves to make motion in animations more clear.

- **Actuator Mass Allocation:** The mass of the joint motors is not modeled as separate floating elements. Instead, these actuator masses are directly integrated into the mass properties of the links as we do not know the weight distribution in the links themselves.
- **Robot Configuration Realism:** The kinematic configuration and joint layout differ from the standard PUMA 560 robot to more closely match current robot designs. However, the baseline physical parameters of the PUMA 560 are utilized to ensure that the simulated 3 DoF manipulator retains realistic, physically feasible proportions and dynamic properties.

3 Simulation

3.1 Elementary Tests

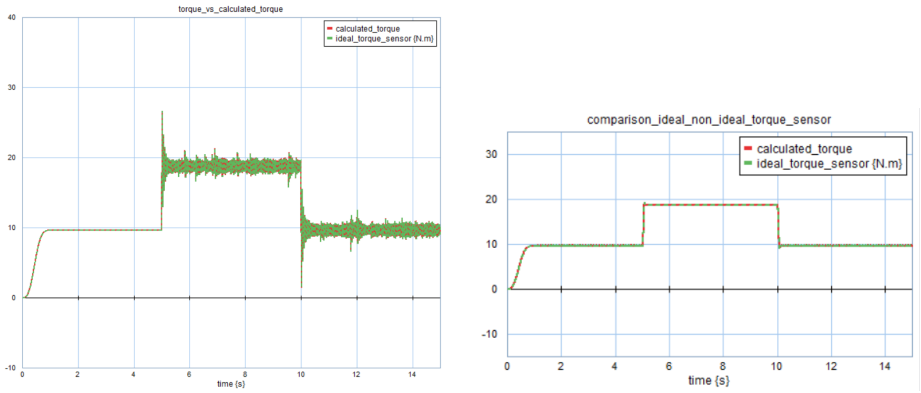
In this section the elementary elements of the model will be verified and validated through simulation. Scenarios such as ideal, nonideal motor, and full robot arm, are discussed which refer to three different .emx files from different stages of the model development. The files of which are respectively called:

- G12_ideal_SEA_with_impedance_control.emx refers to the most ideal scenario where only the joint is modeled with its SEA. The motor here is modelled as an ideal effort source
- G12_nonideal_SEA_with_impedance_control.emx refers to the scenario where there is no dynamic robot arm, but the motor is fully modelled.
- Finally, G12_FULL_IMPLEMENTATION_PUMA_derivative.emx contains the actual full model with everything included.

It is important to keep in mind that model parameters have changed over time and that the only model parameters with guaranteed final model parameters is the G12_FULL_IMPLEMENTATION_PUMA_derivative.emx model. For any of the other models the parameters might be different, but they can still serve as verification for the concepts that are used. The other model files are simply added such that plots can be recreated if desired.

3.1.1 SEA

In order to perform verification of the working of an SEA component, it was first tested in an isolated environment. As seen in Figure 9a, the torque computed with the series elastic element was compared to a ideal motor, where it can be seen that it is the exact same as the one produced by an ideal effort sensor. Looking further into a non-ideal case, from Figure 9b, the losses inside of the motor have been modeled. The SEA element continues to properly track the full output of the motor without deviation.



(a) SEA ideal model (b) SEA non-ideal model

Figure 9: Comparison between the ideal and non-ideal SEA models.

After the SEA component was properly tested on its own, it is now time to verify its behaviour inside the actual model. The simulation in Figure 10 shows the calculated torque perform with a constant and an oscillatory input. There is no deviation from the ideal situation as expected the values are of the calculated and ideal torque are the exact same, validating the working of the SEA element.

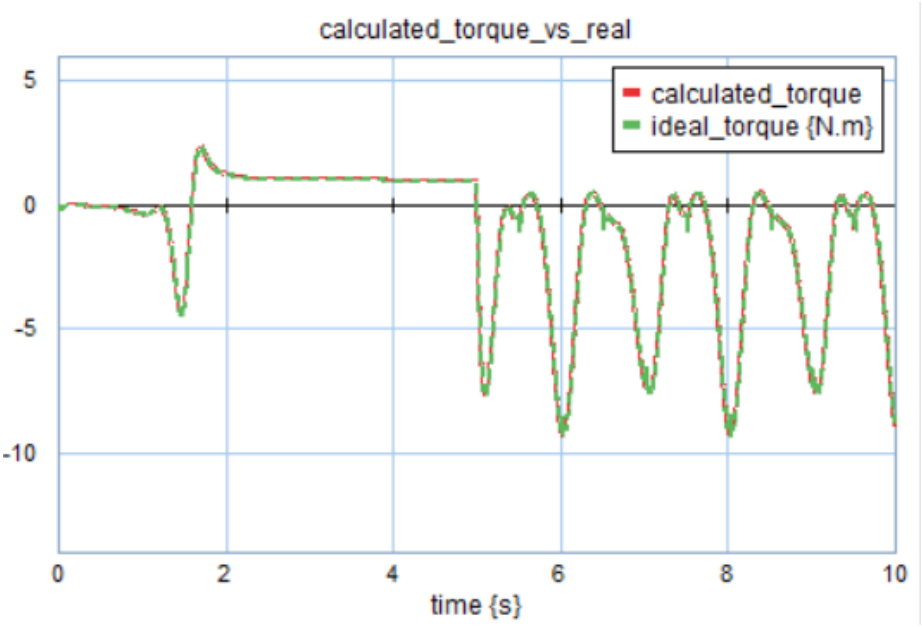


Figure 10: SEA implemented inside the full model.

3.1.2 Torque Control

Moving further on, the torque controller's output, for both current and torque, are verified to be working. In Figures 11 and 12, the outputs of the controller

are compared against the non-ideal motor, with a disturbance implemented after 5 seconds. In the current plot, small oscillations can be seen in the current plot when the constant value is supposed to be reached which make sense as these models still implement the PID controller with a non-zero derivative term. But given that these oscillations did not have a significant effect on the torque controller we can state that this verifies the functionality of the cascaded controllers.

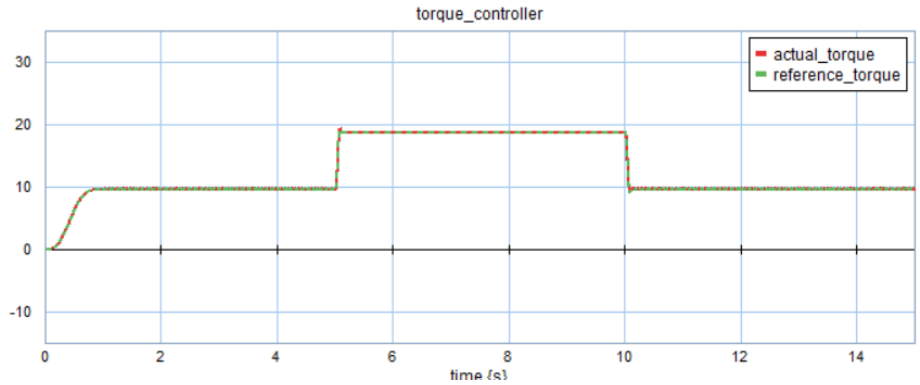


Figure 11: Torque output against an non-ideal motor output.

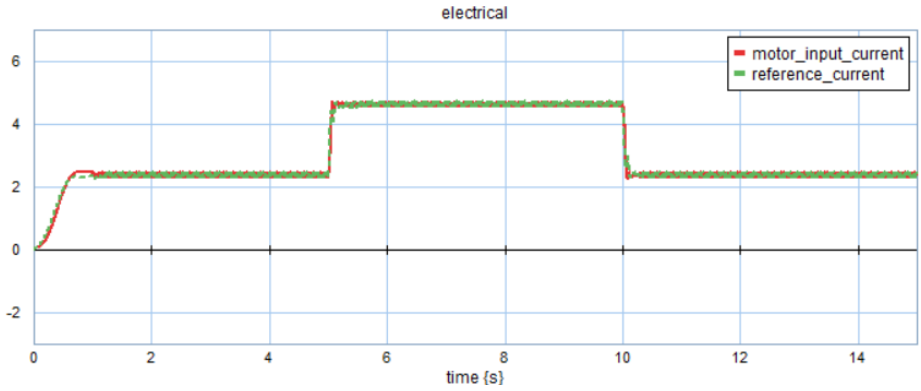


Figure 12: Current output against an non-ideal motor output.

After completing the verification tests for the torque control, it was implemented inside of the full model to perform a validation check for both torque and current. In this model the D element of the PID controller was removed in an attempt to remove the oscillations. Since each of the joint has its own trajectory, all three controllers will be checked. Looking at Figures 13 and 14, the shape of the reference signals is preserved with little to no deviation from the reference values and no additional overshoot, validating the correct working of the cascaded controllers.

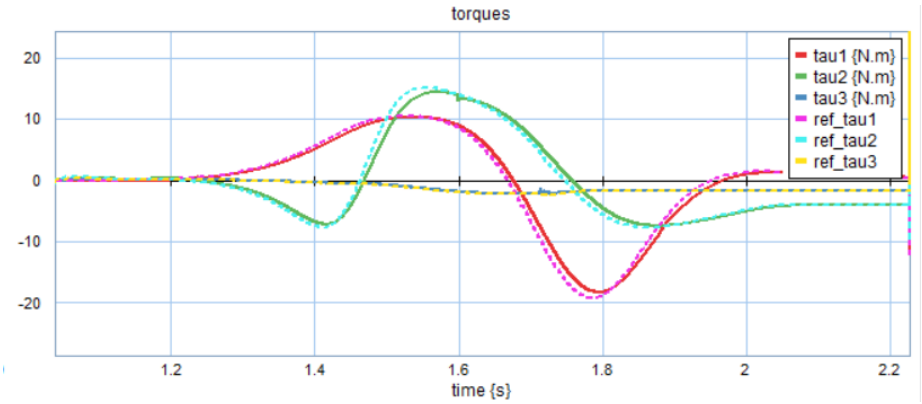


Figure 13: Torque output against the reference torque inside the full model.

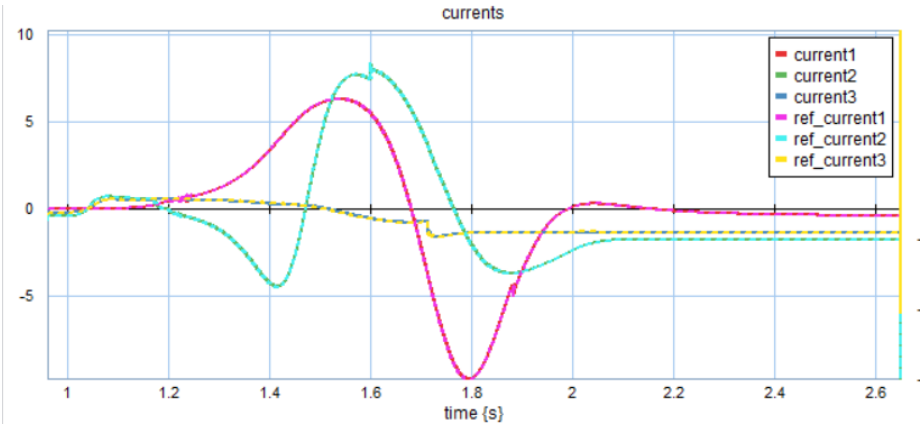
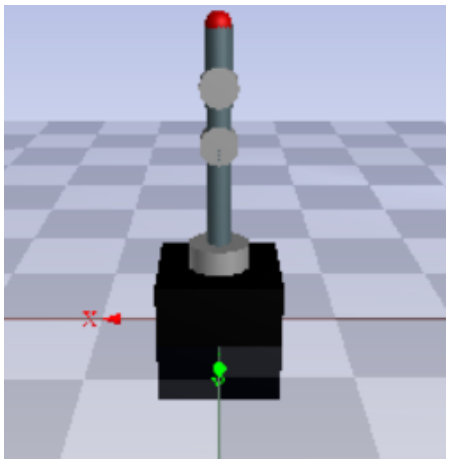


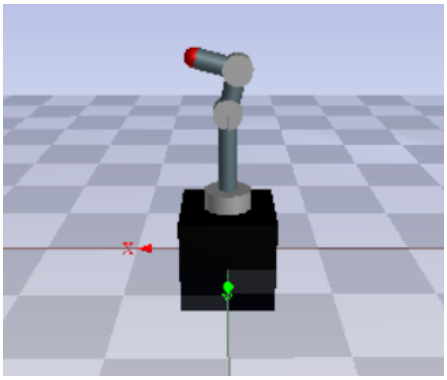
Figure 14: Current output against the reference current inside the full model.

3.1.3 Manipulator modelling

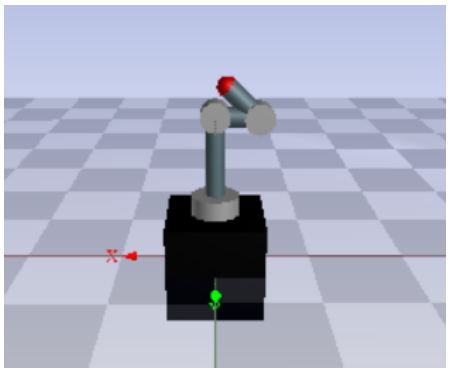
To test the validity of the robot arm model produced by the 3D mechanics toolbox, all of the inputs to the joints were turned off, and a small disturbance was applied to knock the arm over. If the Jacobian matrix and the joint limits are correctly designed, the arm should quickly fall down, with the joints not passing through the segments. Since the first joint rotates around the z axis, no motion should be observed there. As seen in Figure 15, snapshots of the process have been put in chronological order to showcase the full movement. The joints respect the previously mentioned limit, but the end effector, for which no collision behvaviour had been implemented, is seen passing through one of the links. The test satisfies the verification requirement from the start, as no links rotate more that they are allowed, and no additional overlap is seen. The speeds that the robot arm achieves also seem to be realistic these speeds are however better seen when running the model and playing the video. The correct speeds and correct joint limit behaviour validates the model. The trajectory of the end effector can additionally be observed in Figure 16.



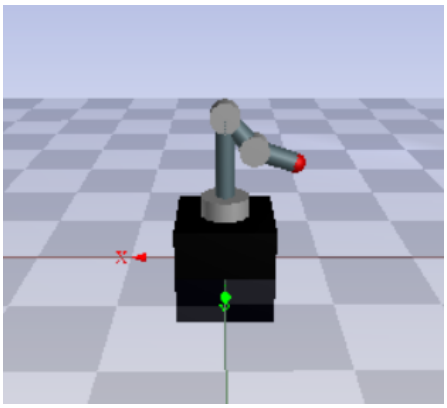
(a) $t = 0$ s



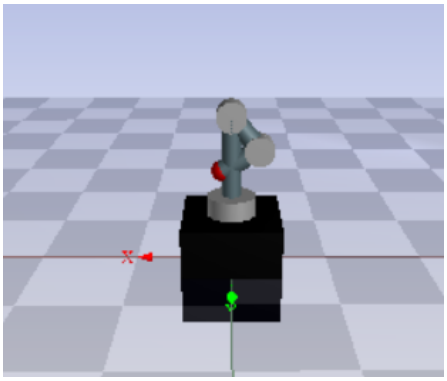
(b) $t = 1.5$ s



(c) $t = 2.2$ s



(d) $t = 2.4$ s



(e) $t = 2.6$ s

Figure 15: Model falling down, with no intput to the joints.

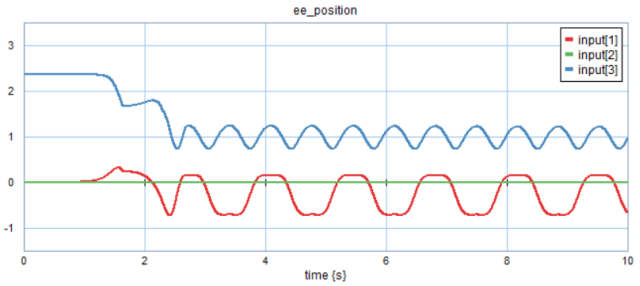


Figure 16: Motion of the end effector position while falling down.

3.1.4 Gravity Compensation

In a real world scenario, robotic arms with gravity compensation are expected to be able to support their own weight while holding a static pose. As such the robot has been shown in Figure 17 to be able to maintain a static pose while under the effect of gravity.

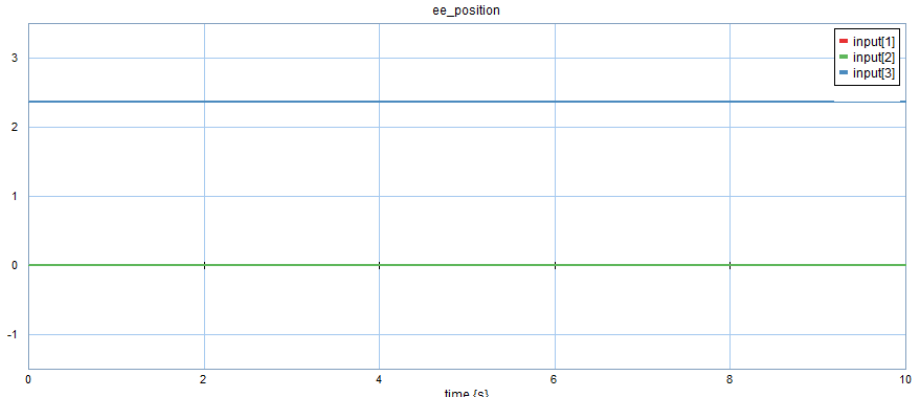


Figure 17: End effector position plot for the system that is only gravity compsensated.

3.1.5 Impedance control

With all other components of the full model tested and validated, it is now time to take a look at the impedance controller. As done above, the first test will be in an isolated case, with an ideal refernce signal to compete against. A beam of 1m with weight of 2kg at 0.5m is used as a body to verify the results. Gravity is applied to this beam at all times during the simulation. There is also a force of 10N which is applied at the tip of a 1 m beam, pointing downward (in the same direction as gravity), at the 5 second mark, to act as a disturbance. In Figure 18, as well as in 19 where the losses of the motor are implemented, the shape of the reference signal is maintained, while at a slight offset down. For the ideal case, a virtual stiffness of $57.8 N/m$ was used, that differs from the stiffnesses of $200 N/m$ used in the full model. The latter were achieved through tuning of the system. Another difference comes from the difference in input between the

isolated case and the full model. In the isolated case, a difference of angles was used, while for the full model each coordinate of the end effector was compared with the trajectory input signals. This discrepancy was done to steamline the validation test of the impedance control. With a moment of $M \approx 19.81$ and a stifness of $K_\theta = 57.8N/m$ there should be a deflection of 0.34 rad which is about 20 degrees. Which is almost exactly what can be seen as well. Rather, a slightly smaller deflection is observed of 0.33 rad is observed. This is a slightly smaller deflection than 0.34 which is because gravity is always downwards, but because of the deflection this is no longer exactly applied in the radial direction. And only a percentage of that gravity is still in the radial direction.

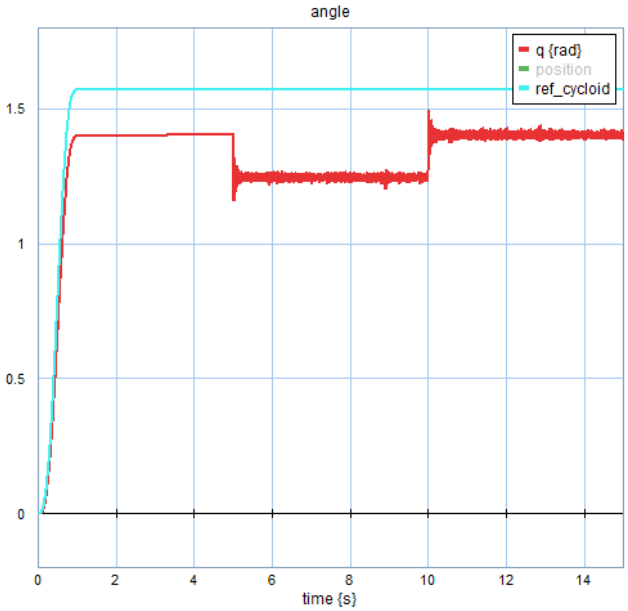


Figure 18: Output of the impedance controller in an ideal model.

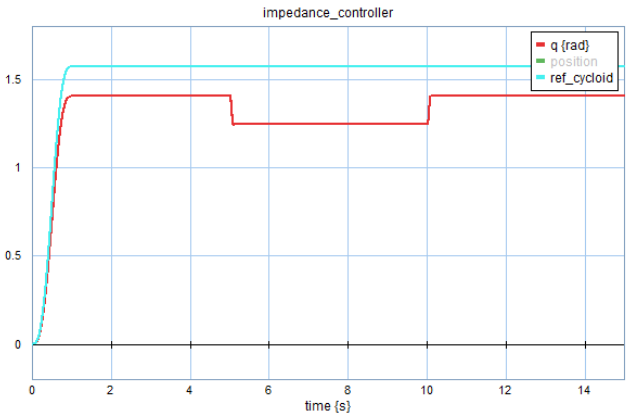


Figure 19: Output of the impedance controller in a non-ideal model.

After the validation tests, the impedance controller was implemented on its own in the complete system. The first test done was without the inclusion of the gravity compensation term, to see the performance alone. Values used for these tests were $200N/m$ for the stiffness of all impedance controllers.The disturbance that's placed here is $[Fx, Fy, Fz] = [10, 20, -30]$ which serves to test each cartesian direction of the end effector which covers cases like a load on the end effector (in this case that load would be about 3kg), but also general push/pull interaction with the end effector. The expected deflection in each direction is $[dx, dy, dz] = \mathbf{F}./\mathbf{K}_{stiff,virtual} = [\frac{10}{200}, \frac{20}{200}, \frac{-30}{200}] = [0.05, 0.1, -0.15]$. The validation test was satisfactory, as the shape of the reference signal is mostly maintained and the deflections are within a margin of error. The small discrepancy can be attributed to the system trying to compensate for gravity's influence on the motion without having a proper way to do so.

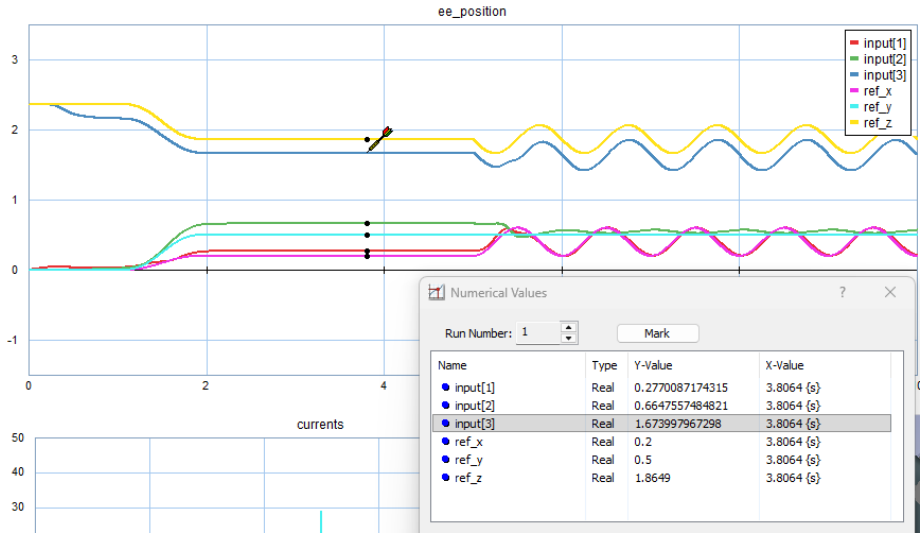


Figure 20: Output of the impedance controller in the full model, without gravity compensation.

In Figure 21, the compensation term is added back to become the full system. We first run this simulation without any disturbance. The system is a bit slow at the beginning but shows almost perfect matching in both the constant phase and oscillatory phase, slightly less so in the oscillatory phase. All of which makes sense and is expected as the dynamics are slowed down due to spring like behaviour and cause the "end of the spring" which is the end effector to lag behind the desired position during motion.

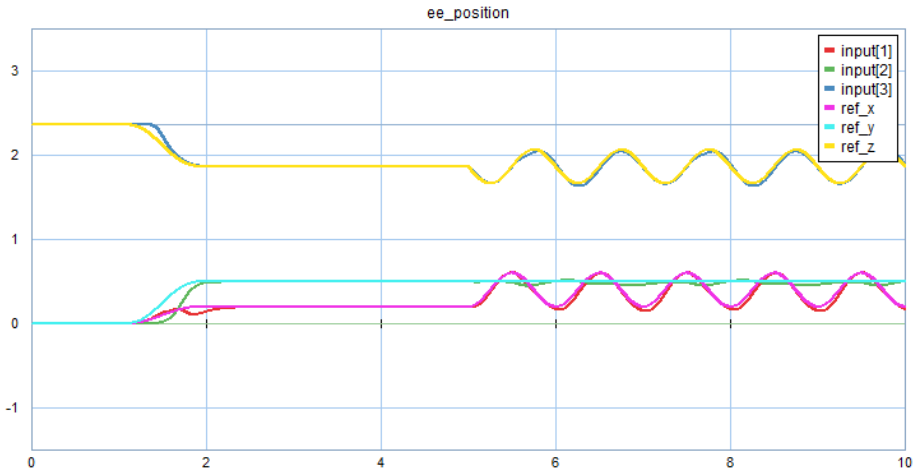


Figure 21: Output of the impedance controller in the full model, with gravity compensation.

When the same disturbance as before of $[Fx, Fy, Fz] = [10, 20, -30]$ was added to the full system, and the stiffnesses of the controllers were equal to $200N/m$, it can be seen in Figure 22 that the end effector reaches exactly the deflection that we expect of $[dx, dy, dz] = \mathbf{F}./\mathbf{K}_{stiff,virtual} = [\frac{10}{200}, \frac{20}{200}, \frac{-30}{200}] = [0.05, 0.1, -0.15]$ up to a margin of error of 0.0005m. Which could even be due to the modelled but unaccounted for end effector weight of 1 gram.

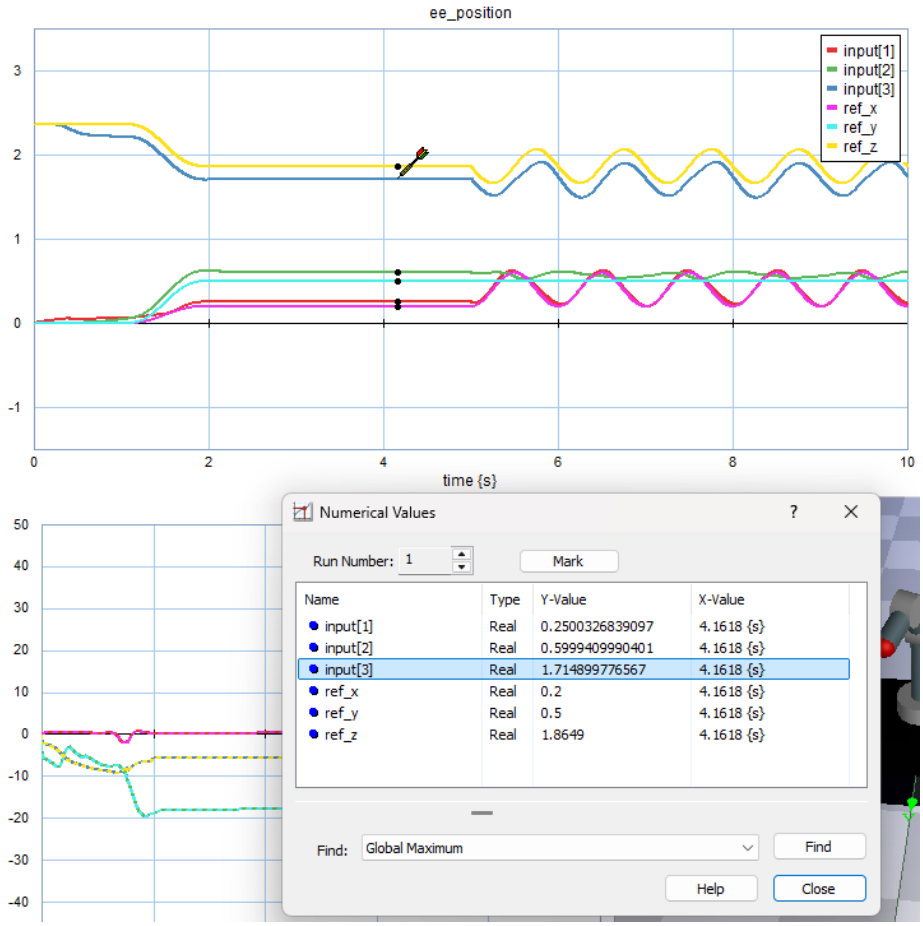


Figure 22: Output of the impedance controller in the full model, with gravity compensation and disturbance applied.

It can also be proven that the stiffness parameters can be changed to different values if the situations calls for that and also the stiffness parameters are truly indepedent of eachother. This is tested by changing the x and y stiffness parameters to $400N/m$ while applying the same disturbance as before. Here we expect deflections of [0.025, 0.05, -1.5]. The results can be seen in Figure 23 show once again exactly the expected changed deflection.

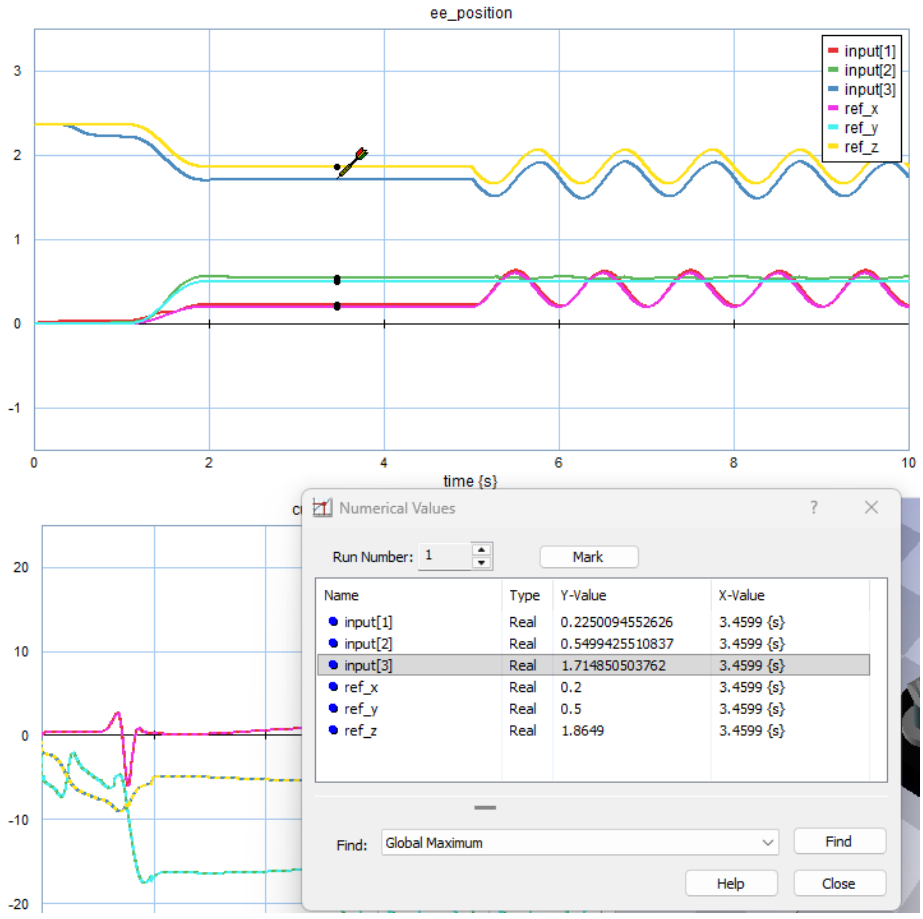


Figure 23: Output of the impedance controller in the full model, with gravity compensation and stiffnesses in x and y increased.

4 Discussion & Conclusion

Each piece of the full model was shown to pass both validation and verification tests and behave as desired. The arm was shown maintaining a static pose without any influence of the controllers, proving the gravity compensation was done corectly. The SEA also behaved correctly as a torque sensor, perfectly computing the torque generated in both an isolated case and in the full implementation. Good tracking was also observed in the cascaded current and torque controllers, only having a small deviation from the desired response.

The impedance controller showed expected behaviour during all possible test cases with an accuracy of up to ± 0.0005 in the gravity compensated case. The full implementation of the robot arm showed great results for both a constant input as well as an oscillatory input.

Collision detection of the end effector is one of the missing elements of this model which we would have liked to implement in some fashion as well as maybe testing

what happens when the robot interacts with a rigid object such as a wall.

5 AI Statement

During the preparation of this work, the authors used Grammarly, ChatGPT, Gemini, and Overleaf Writefull to help with spelling and grammar checks as well as to translate some words from Spanish to English. They have also been used to create tables more easily in latex. After using this tool/service, the author reviewed and edited the content as needed and took full responsibility for the content of the work.

References

[1] *AK10-9 V2.0 KV60*. URL: https://www.cubemars.com/product/ak10-9-v2-0-kv60-robotic-actuator.html.

[2] *Programmable Universal Machine for Assembly*. Sept. 2025. URL: https://en.wikipedia.org/wiki/Programmable_Universal_Machine_for_Assembly.

[3] *UR7e*. URL: https://www.universal-robots.com/products/ur7e/.